

# Cours MVSI

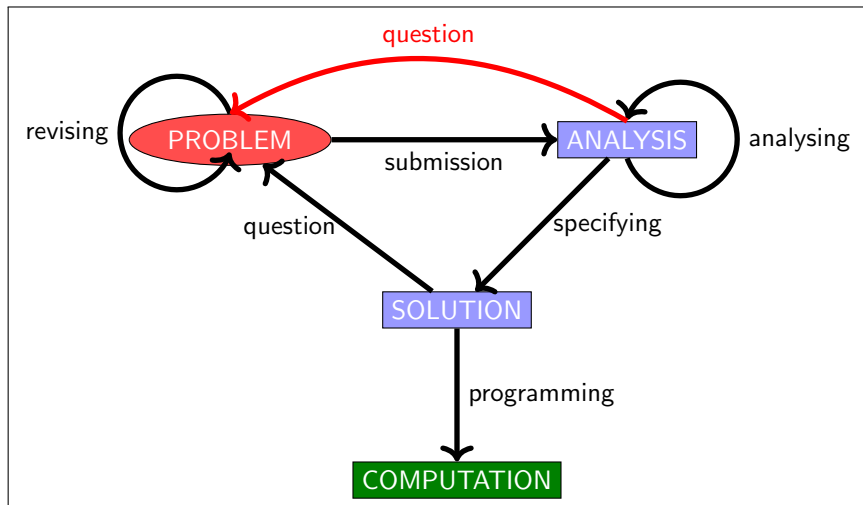
## Modélisation et Vérification des Systèmes Informatiques

# Sommaire du cours

Dominique Méry  
Telecom Nancy, Université de Lorraine  
(7 octobre 2026 at 2:08 P.M.)

- ① Tracking bugs in C codes
- ② Introduction by Example
  - Detecting overflows in computations
  - Computing the velocity of an aircraft on the ground
- ③ Verification of program properties
- ④ Topics of course

## Problem versus Solution



## Listing 1 – Bug bug0

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int x, y;
    // Seed the random number generator with the current time
    srand(time(NULL));
    // Generate a random number between 1 and 100
    x = rand() % 100 + 1;
    // Perform some calculations
    y = x / (100 - x);
    printf("x=%d--and--y=%d\n", x, y);
    return 0;
}
```

## Listing 2 – Bug bug0

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int x, y;
    // Seed the random number generator with the current time
    srand(time(NULL));
    // Generate a random number between 1 and 100
    x = rand() % 100 + 1;
    // Perform some calculations
    y = x / (100 - x);
    printf("x=%d - - and -y=%d\n", x, y);
    return 0;
}
```

**bug0.c prints x=some number and y=some other number**

## Listing 3 – Bug bug00

```
// Heisenbug
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
//
int main() {
    int x, y, i=0;
    for (i = 0; i <= 1000000; i++) {
        // Seed the random number generator with the current time
        srand(time(NULL));
        // Generate a random number between 1 and 100
        x = rand() % 100 + 1;
        printf("Result: -x=-%d\n", x);
        // Perform some calculations
        y = x / (100 - x);
        printf("Result: -i=%d--and-y=%d\n", i, y);
    }
    return 0;
}
```

## Listing 4 – Bug bug00

```
// Heisenbug
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
//
int main() {
    int x, y, i=0;
    for (i = 0; i <= 1000000; i++) {
        // Seed the random number generator with the current time
        srand(time(NULL));
        // Generate a random number between 1 and 100
        x = rand() % 100 + 1;
        printf("Result: -x=-%d\n", x);
        // Perform some calculations
        y = x / (100 - x);
        printf("Result: -i=%d--and-y=%d\n", i, y);
    }
    return 0;
}
```

**bug00.c prints ... i=some  
number and y=some other number  
...**

## Listing 5 – Bug bug000

```
// Heisenbug
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <limits.h>

int main() {
    int x, y, i=0;

    for (i = 0; i <= INT_MAX; i++) {
        // Seed the random number generator with the current time
        srand(time(NULL)+i);

        // Generate a random number between 1 and 100
        x = rand() % 100 + 1;

        printf("Result: -x=-%d\n", x);
        // Perform some calculations
        y = x / (100 - x);

        printf("Result: -i=%d -x=-%d -and -y=%d\n", i, x, y);
    }

    return 0;
}
```



## Listing 6 – Bug bug000

```
// Heisenbug
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <limits.h>

int main() {
    int x, y, i=0;

    for (i = 0; i <= INT_MAX; i++) {
        // Seed the random number generator with the current time
        srand(time(NULL)+i);

        // Generate a random number between 1 and 100
        x = rand() % 100 + 1;

        printf("Result: -x=-%d\n", x);
        // Perform some calculations
        y = x / (100 - x);

        printf("Result: -i=%d -x=-%d -and -y=%d\n", i, x, y);
    }

    return 0;
}
```

Result: x= ??

Result: i=?? x=?? and y=??

- ▶ RTE means Run Time Error
- ▶  $x = \text{rand}() \% 100 + 1$  and  $y = x / (100 - x)$
- ▶  $\text{rte}(x = \text{rand}() \% 100 + 1) \stackrel{\text{def}}{=} \text{rte}(x) \wedge \text{rte}(\text{rand}()) \wedge \text{rte}(\text{rand}() \% 100 + 1)$
- ▶  $\text{rte}(y = x / (100 - x)) \stackrel{\text{def}}{=} \text{ret}(y) \wedge \text{rte}(x) \wedge 100 \neq x$

## RTE

The value of  $x$  may be  $100$  and a sequence of test can not get  $x$  with the value  $100$ .

## Listing 7 – Bug bug1

```
#include <stdio.h>
#define n 5
int main() {
    int numbers[n] = {1, 2, 3, 4, 5};
    int sum = 0;

    // Attempt to calculate the sum of numbers in the array
    for (int i = 0; i <= n; i++) {
        sum += numbers[i];
    };
    int val = n*(n+1)/2;
    printf(" Iteratively -computed -sum=%d -and -direct -comutation -val=%d\n" , sum, val);
    return 0;
}
```

## Listing 8 – Bug bug1

```
#include <stdio.h>
#define n 5
int main() {
    int numbers[n] = {1, 2, 3, 4, 5};
    int sum = 0;

    // Attempt to calculate the sum of numbers in the array
    for (int i = 0; i <= n; i++) {
        sum += numbers[i];
    };
    int val = n*(n+1)/2;
    printf(" Iteratively -computed- sum=%d -and- direct -comutation- val=%d\n", sum, val);
    return 0;
}
```

Iteratively computed sum=16 and  
direct comutation val=15

## Listing 9 – Bug bug2

```
#include <stdio.h>

int main() {
    int x = 5;
    int y = 3;

    // Bug 1: Incorrect variable in the printf statement
    printf("The-value-of-x-is:-%d\n", y);

    // Bug 2: Infinite loop
    while (x > 0) {
        printf("x-is-greater-than-0\n");
    }

    return 0;
}
```

## Listing 10 – Bug bug2

```
#include <stdio.h>

int main() {
    int x = 5;
    int y = 3;

    // Bug 1: Incorrect variable in the printf statement
    printf("The-value-of-x-is:-%d\n", y);

    // Bug 2: Infinite loop
    while (x > 0) {
        printf("x-is-greater-than-0\n");
    }

    return 0;
}
```

**Infinite loop . . .**



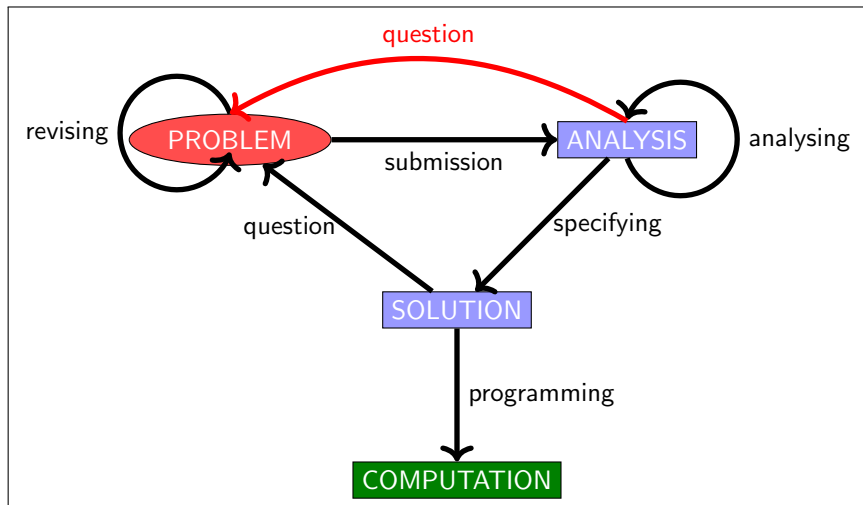
## Listing 12 – Bug bug7

```
#include <stdio.h>
#include <limits.h>
int average(int a, int b)
{
    return ((a+b)/2);
}

int main()
{
    int x, y;
    x=INT_MAX; y=INT_MAX;
    printf(" Average -- for %d - and %d - is %d\n", x, y,
        average(x, y));
    return 0;
}
```

Average for 2147483647 and  
2147483647 is -1





### Listing 13 – Function average

```
#include <stdio.h>
#include <limits.h>
int average(int a,int b)
{
    return((a+b)/2);
}

int main()
{
    int x,y;
    x=INT_MAX;y=INT_MAX;
    printf(" Average - - for -%d - and -%d - is -%d\n" ,x,y ,
           average(x,y));
    return 0;
}
```

Execution produces a result

Average for 2147483647 and 2147483647 is -1

### Execution produces a result

Average for 2147483647 and 2147483647 is -1

### Using frama-c produces a required annotation

```
int average(int a, int b)
{
    int __retres;
    /*@ assert rte: signed_overflow: -2147483648 <= a + b; */
    /*@ assert rte: signed_overflow: a + b <= 2147483647; */
    __retres = (a + b) / 2;
    return __retres;
}
```

Listing 14 – Function average.....

```
#include <stdio.h>
#include <limits.h>
/*@ requires 0 <= a;
    requires a <= INT_MAX ;
    requires 0 <= b;
    requires b <= INT_MAX ;
    requires 0 <= a+b;
    requires a+b <= INT_MAX ;
    ensures \result <= INT_MAX;

*/
int average(int a,int b)
{
    return((a+b)/2);
}

int main()
{
    int x,y;
    x=INT_MAX / 2;y=INT_MAX / 2;
    // printf("Average for %d and %d is %d\n",x,y,
    // );
    return average(x,y);
}
```



- ▶ Estimated ground velocity of the aircraft should be available only if it is within 3 km/hr of the true velocity at some moment within past 3 seconds

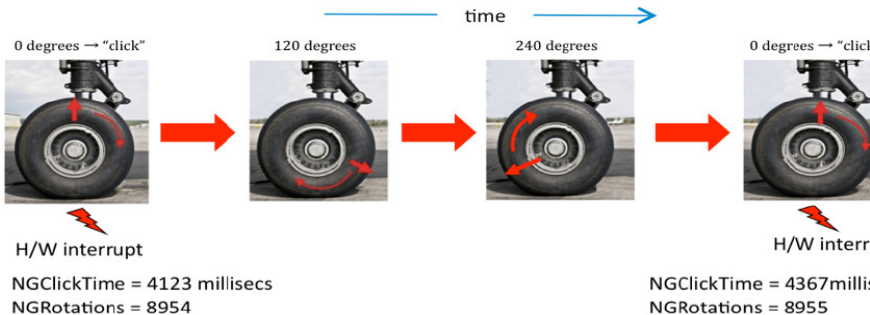
## Characterization of a System (I)

- ▶ **NG velocity system :**
  - **Hardware :**
    - ▶ *Electro-mechanical sensor* : detects rotations
    - ▶ *Two 16-bit counters* : Rotation counter, Milliseconds counter
    - ▶ *Interrupt service routine* : updates rotation counter and stores current time.
  - **Software :**
    - ▶ *Real-time operating system* : invokes update function every 500 ms
    - ▶ *16-bit global variable* : for recording rotation counter update time
    - ▶ *An update function* : estimates ground velocity of the aircraft.
- ▶ **Input data available to the system :**
  - *time* : in milliseconds
  - *distance* : in inches
  - *rotation angle* : in degrees
- ▶ **Specified system performs velocity estimations in *imperial* unit system**
- ▶ **Note :** expressed functional requirement is in *SI* unit system (km/hr).





## Sample Velocity Estimation



WHEEL\_DIAMETER = 22 inches  
PI = 3.14

12 inches/foot  
5280 feet/mile

estimatedGroundVelocity = distance travel/elapsed time  
=  $((3.14 * 22) / (12 * 5280)) / ((4367 - 4123) / (1000 * 3600))$   
= 16 mph

### Safety Property

- ▶ Storing the number of *NGClick* in a n-bit variable *VNGClick*
- ▶ Integers are denoted by the set *Int* and is simply defined by the interval  $Int \hat{=} INT\_MIN..INT\_MAX$ .
- ▶ Safety requirement :  
*The value of VNGClick is always in the range of implementation Int or equivalently  $VNGClick \in Int$*
- ▶  $Length = \pi \cdot diameter \cdot VNGClick$  (mathematical property)
- ▶  $Length \leq 6000$  (domain property)
- ▶  $\pi \cdot diameter \cdot VNGClick \leq 6000$
- ▶  $VNGClick \leq 6000 / (\pi \cdot diameter)$
- ▶ if  $n=8$ , then  $2^7 - 1 = 127$  and  
 $6000 / (\pi \cdot [22, inch]) = 6000 / (\pi \cdot 55, 88) = 6000 / (3, 24 \cdot [55, 88, cm]) = 6000 / (3, 24 \cdot 0.5588) \approx 3419$  and the condition of safety can not be satisfied in any situation.
- ▶ if  $n=16$ , then  $2^{15} - 1 = 65535$  and  $6000 / (\pi \cdot [22, inch]) \approx 3419$  and the condition of safety can be satisfied in any situation since

### Safety Property

- ▶ Storing the number of `NGClick` in a n-bit variable `VNGClick`
- ▶ Integers are denoted by the set *Int* and is simply defined by the interval  $Int \hat{=} INT\_MIN..INT\_MAX$ .
- ▶ Safety requirement :  
*The value of `VNGClick` is always in the range of implementation *Int* or equivalently  $VNGClick \in Int$*

$$RTE\_VNGClick : 0 \leq vNGClick \leq INT\_MAX \quad (1)$$

- ▶ The current value of `VNGClick` is always bounded by the two values 0 and `INT_MAX`.



- ▶ Typing Properties using Typechecker (see for instance functional programming languages as ML, CAML, OCAML, ...)
- ▶ Invariance and safety (*A nothing bad will happen !*) properties for a program P :
  - Transformation of P into a relational model M simulating P
  - Expression of safety properties :
 
$$\forall s, s' \in \Sigma. (s \in \text{Init}_S \wedge s \xrightarrow{*} s') \Rightarrow (s' \in A).$$
  - Definition of the set of reachable states of P using M :
 
$$\text{REACHABLE}(M) = \text{Init}_S \cup \bigcup \text{[REACHABLE}(M)]$$
  - Main property of REACHABLE(M) :  $\text{REACHABLE}(M) \subseteq A$
  - Characterization of REACHABLE(M) :
 
$$\text{REACHABLE}(M) = \text{FP}(\text{REACHABLE}(M))$$

- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  :



- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : undecidable ... no program is able to prove it automatically!
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  :



- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : undecidable ... no program is able to prove it automatically!
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : possible when restrictions over the set of states is possible (finite set of states)



- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : undecidable ... no program is able to prove it automatically!
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : possible when restrictions over the set of states is possible (finite set of states)
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : possible for some classes of systems and with some tools.

- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : undecidable ... no program is able to prove it automatically !
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : possible when restrictions over the set of states is possible (finite set of states)
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : possible for some classes of systems and with some tools.
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : changing the domain and solving in another domain as abstract interpretation if making possible

- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : undecidable ... no program is able to prove it automatically !
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : possible when restrictions over the set of states is possible (finite set of states)
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : possible for some classes of systems and with some tools.
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : changing the domain and solving in another domain as abstract interpretation if making possible
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  :

- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : undecidable ... no program is able to prove it automatically !
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : possible when restrictions over the set of states is possible (finite set of states)
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : possible for some classes of systems and with some tools.
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : changing the domain and solving in another domain as abstract interpretation if making possible
- ▶ Proving automatically  $\text{REACHABLE}(M) \subseteq A$  : approximating semantics of programs



- A problem  $x \in P$  is generally stated by the function  $\chi_{x \in P}$  where  $\chi_{x \in P}(u) = 1$ , if  $P(u)$  is true and  $\chi_{x \in P}(u) = 0$ , if  $P(u)$  is false :
- Problem 1 :  $x \in 0..n$  where  $n \in \mathbb{N}$
  - Problem 1 :  $w \in \mathcal{L}(G)$  where  $G$  is a grammar over the finite set of alphabet symbols  $\Sigma$  and  $\mathcal{L}(G) \subseteq \Sigma^*$ .



- ▶ A problem  $x \in P$  is generally stated by the function  $\chi_{x \in P}$  where  $\chi_{x \in P}(u) = 1$ , if  $P(u)$  is true and  $\chi_{x \in P}(u) = 0$ , if  $P(u)$  is false :
  - Problem 1 :  $x \in 0..n$  where  $n \in \mathbb{N}$
  - Problem 1 :  $w \in \mathcal{L}(G)$  where  $G$  is a grammar over the finite set of alphabet symbols  $\Sigma$  and  $\mathcal{L}(G) \subseteq \Sigma^*$ .
- ▶ A problem  $x \in P$  is decidable, when the function  $\chi_{x \in P}$  is computable or more precisely the function can be computed by a program



## Implicite versus explicite

- Ecrire  $101 = 5$  peut avoir une signification





- ▶ Un programme  $P$  *produit* des résultats à partir de données en accord avec une sémantique :
  - STATES est l'ensemble de tous les états de  $P$  :  $STATES = x \rightarrow \mathbb{Z}$  où  $x$  désigne les variables de  $P$ .
  - $s_0$  et  $s_f$  deux états de STATES :  $\mathcal{D}(P)(s_0) = s_f$  signifie que  $P$  est exécuté à partir d'un état  $s_0$  et produit un état  $s_f$ .
  - Pour un état  $s$  de  $P$  courant, on notera  $s(x) = x$  pour distinguer la valeur de la variable  $x$  et sa valeur courante en  $s$  :

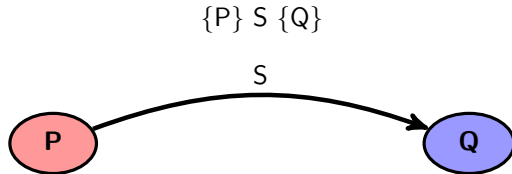


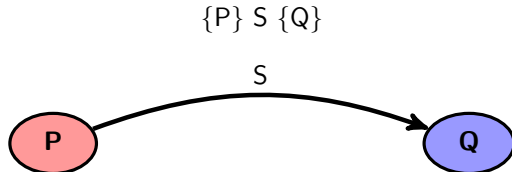




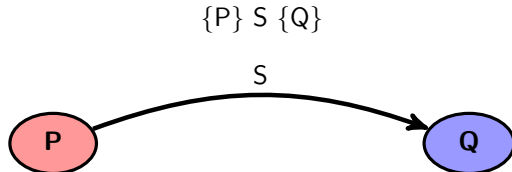




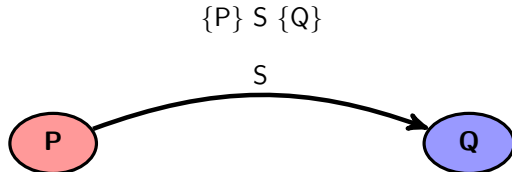




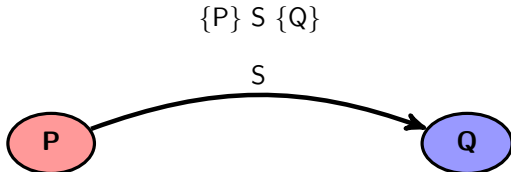
►  $\{P\} S \{Q\}$  : *asserted program*



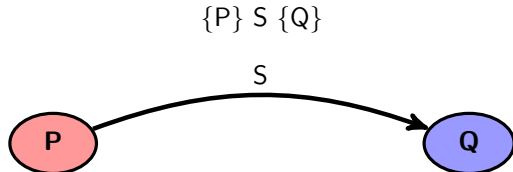
- ▶  $\{P\} S \{Q\}$  : *asserted program*
- ▶  $P \Rightarrow WP(S)(Q)$  : *logical formula*



- ▶  $\{P\} S \{Q\}$  : *asserted program*
- ▶  $P \Rightarrow WP(S)(Q)$  : *logical formula*
- ▶  $C(S) \vdash P \Rightarrow WP(S)(Q)$  : *logical formula*

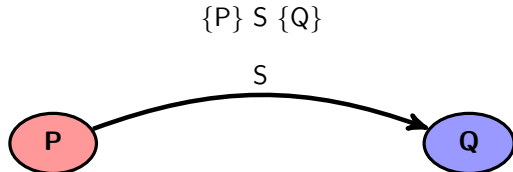


- ▶  $\{P\} S \{Q\}$  : *asserted program*
- ▶  $P \Rightarrow WP(S)(Q)$  : *logical formula*
- ▶  $C(S) \vdash P \Rightarrow WP(S)(Q)$  : *logical formula*
- ▶  $SP(S)(P) \Rightarrow Q$  : *logical formula*

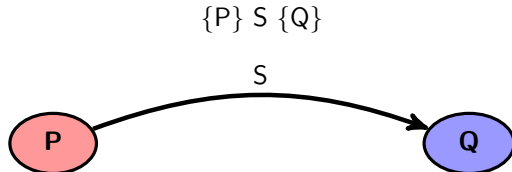


- ▶  $\{P\} S \{Q\}$  : *asserted program*
- ▶  $P \Rightarrow WP(S)(Q)$  : *logical formula*
- ▶  $C(S) \vdash P \Rightarrow WP(S)(Q)$  : *logical formula*
- ▶  $SP(S)(P) \Rightarrow Q$  : *logical formula*
- ▶  $C(S) \vdash SP(S)(P) \Rightarrow Q$  : *logical formula*





- ▶  $\{P\} S \{Q\}$  : *asserted program*
- ▶  $P \Rightarrow WP(S)(Q)$  : *logical formula*
- ▶  $C(S) \vdash P \Rightarrow WP(S)(Q)$  : *logical formula*
- ▶  $SP(S)(P) \Rightarrow Q$  : *logical formula*
- ▶  $C(S) \vdash SP(S)(P) \Rightarrow Q$  : *logical formula*



- ▶  $\{P\} S \{Q\}$  : *asserted program*
- ▶  $P \Rightarrow WP(S)(Q)$  : *logical formula*
- ▶  $C(S) \vdash P \Rightarrow WP(S)(Q)$  : *logical formula*
- ▶  $SP(S)(P) \Rightarrow Q$  : *logical formula*
- ▶  $C(S) \vdash SP(S)(P) \Rightarrow Q$  : *logical formula*

## Predicate Transformer

$WP(S)(Q)$  is the Weakest-Precondition of  $S$  for  $Q$  and is a predicate transformer but  $WP(S)(.)$  is not a computable function over the set of predicates.





- ▶ Découpage de l'unité : 10 cours de 2 h
- ▶ Contenu :
  - 😊 Principes de modélisation des systèmes informatiques : systèmes de transition



- ▶ Découpage de l'unité : 10 cours de 2 h
- ▶ Contenu :
  - 😊 Principes de modélisation des systèmes informatiques : systèmes de transition
  - 😊 Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - 😊 Modélisation de propriétés de systèmes

- ▶ Découpage de l'unité : 10 cours de 2 h
- ▶ Contenu :
  - 😊 Principes de modélisation des systèmes informatiques : systèmes de transition
  - 😊 Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - 😊 Modélisation de propriétés de systèmes
  - 😊 Analyse des programmes



- ▶ Découpage de l'unité : 10 cours de 2 h
- ▶ Contenu :
  - 😊 Principes de modélisation des systèmes informatiques : systèmes de transition
  - 😊 Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - 😊 Modélisation de propriétés de systèmes
  - 😊 Analyse des programmes
  - 😊 Vérification de propriétés de systèmes avec un model checker TLC

- ▶ Découpage de l'unité : 10 cours de 2 h
- ▶ Contenu :
  - 😊 Principes de modélisation des systèmes informatiques : systèmes de transition
  - 😊 Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - 😊 Modélisation de propriétés de systèmes
  - 😊 Analyse des programmes
  - 😊 Vérification de propriétés de systèmes avec un model checker TLC
  - 😊 Vérification de propriétés de systèmes avec un outil de preuve Rodin

- ▶ Découpage de l'unité : 10 cours de 2 h
- ▶ Contenu :
  - 😊 Principes de modélisation des systèmes informatiques : systèmes de transition
  - 😊 Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - 😊 Modélisation de propriétés de systèmes
  - 😊 Analyse des programmes
  - 😊 Vérification de propriétés de systèmes avec un model checker TLC
  - 😊 Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - 😊 Vérification de propriétés de systèmes avec un analyseur Frama-c



- ▶ Découpage de l'unité : 10 cours de 2 h
- ▶ Contenu :
  - 😊 Principes de modélisation des systèmes informatiques : systèmes de transition
  - 😊 Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - 😊 Modélisation de propriétés de systèmes
  - 😊 Analyse des programmes
  - 😊 Vérification de propriétés de systèmes avec un model checker TLC
  - 😊 Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - 😊 Vérification de propriétés de systèmes avec un analyseur Frama-c
- ▶ Outils

- ▶ Découpage de l'unité : 10 cours de 2 h
- ▶ Contenu :
  - 😊 Principes de modélisation des systèmes informatiques : systèmes de transition
  - 😊 Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - 😊 Modélisation de propriétés de systèmes
  - 😊 Analyse des programmes
  - 😊 Vérification de propriétés de systèmes avec un model checker TLC
  - 😊 Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - 😊 Vérification de propriétés de systèmes avec un analyseur Frama-c
- ▶ Outils
  - 😊 TLA/TLA<sup>+</sup> avec TLC,

- ▶ Découpage de l'unité : 10 cours de 2 h
- ▶ Contenu :
  - 😊 Principes de modélisation des systèmes informatiques : systèmes de transition
  - 😊 Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - 😊 Modélisation de propriétés de systèmes
  - 😊 Analyse des programmes
  - 😊 Vérification de propriétés de systèmes avec un model checker TLC
  - 😊 Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - 😊 Vérification de propriétés de systèmes avec un analyseur Frama-c
- ▶ Outils
  - 😊 TLA/TLA<sup>+</sup> avec TLC, Frama-c

- ▶ Découpage de l'unité : 10 cours de 2 h
- ▶ Contenu :
  - 😊 Principes de modélisation des systèmes informatiques : systèmes de transition
  - 😊 Propriétés d'un système informatique : sûreté, vivacité, disponibilité, sécurité, dépendabilité
  - 😊 Modélisation de propriétés de systèmes
  - 😊 Analyse des programmes
  - 😊 Vérification de propriétés de systèmes avec un model checker TLC
  - 😊 Vérification de propriétés de systèmes avec un outil de preuve Rodin
  - 😊 Vérification de propriétés de systèmes avec un analyseur Frama-c
- ▶ Outils
  - 😊 TLA/TLA<sup>+</sup> avec TLC, Frama-c



